Lab 2 (Recursion in Prolog)

Recursion in Prolog is when a predicate calls itself to solve a problem step-by-step — just like in other programming languages. But in Prolog, recursion is used to navigate through data like lists, trees, or rule chains.

Why Use Recursion in Prolog?

Because:

- Prolog has no loops (no for, while, etc.)
- You process data (like lists or relationships) by defining a base case and a recursive rule

General Structure of Recursion

```
% Base case: simplest condition
        predicate(Condition) :- ... .

% Recursive case: predicate calls itself
        predicate(X) :-
                some_condition(X),
                predicate(NewX).
```

Every recursive rule in Prolog has **two parts**:

Part	Purpose
Base Case	Stop Recursion
Recursive Case	Reduces problem and calls itself

Example 1: Sum of Natural Numbers

```
% Base case: sum of 0 is 0
sum(0, 0).

% Recursive case: sum(N) = sum(N-1) + N
sum(N, Result) :-
  N > 0,
  N1 is N - 1,
  sum(N1, SubResult),
  Result is SubResult + N.
```

Explanation of the code:

Line 1: Base Case

sum(0, 0).

This is the base case.
It says: the sum of numbers up to 0 is 0.
This stops the recursion — it's the simplest possible version of the problem.

Line 2: Recursive Case (Header)

sum(N, Result) :-

This defines a rule for when N > 0.
It means:
"To calculate the sum up to N, do the following..."

Line 3: Condition

N > 0,

Ensures that recursion only works for positive numbers.
If N = 0, this line fails, so Prolog uses the base case instead**.**

Line 4: Decrease Step

N1 is N - 1,

Calculates N1, which is N - 1.
This is how recursion progresses: make the problem smaller.

Line 5: Recursive Call

sum(N1, SubResult)

Calls the same predicate sum again, but with N1 (a smaller value).
SubResult stores the sum of all numbers from 1 to N-1.

Line 6: Final Computation

Result is SubResult + N.

Adds N to the result of sum(N1, SubResult)
Stores the result in Result

```
% Base case: sum of 0 is 0
sum(0, 0).

% Recursive case: sum(N) = sum(N-1) + N
sum(N, Result) :-
  N > 0,
  N1 is N - 1,
  sum(N1, SubResult),
  Result is SubResult + N.
```

Example Execution

Example: sum(3, Result)

```
sum(3, Result) →
        N > 0
        → N1 = 2
        → call sum(2, SubResult)
sum(2, SubResult) →
        N > 0
        →N1 = 1
        →call  sum(1, SubResult)
sum(1, SubResult)
        N > 0
        → N1 = 0
        → call sum(0, SubResult)
```

What happens inside sum(0, SubResult)?

Prolog matches this clause:
sum(0, 0).
This is a fact, so:
sum(0, 0) succeeds immediately
SubResult is unified with 0
So the call:
sum(0, SubResult)
binds:
SubResult = 0

Unwinding step-by-step:
Step 1: sum(0, 0) returns

No calculation needed — base case says: sum(0, 0).

Step 2: Return to sum(1, SubResult)

N = 1

SubResult = 0 ← from sum(0, SubResult)

Result is SubResult + N → Result is 0 + 1 = 1

Step 3: Return to sum(2, SubResult1)

N = 2

SubResult = 1 ← from sum(1, SubResult)

Result is SubResult + N Result is 1 + 2 = 3

Step 4: Return to sum(3, Result)

N = 3

SubResult = 3 ← from sum(2, SubResult)

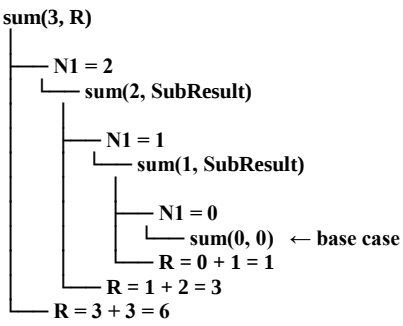
Result is SubResult + N Result is 3 + 3 = 6

So:

?- sum(3, Result).

Result = 6.

Recursive Call Tree for sum(3, Result)



Example 2: Traverse Course Prerequisites

```
prerequisite(cs101, cs102).
prerequisite(cs102, cs201).
prerequisite(cs201, ai301).

% Base case: direct link
course_path(X, Y) :- prerequisite(X, Y).

% Recursive case: indirect link
course_path(X, Y) :-
   prerequisite(X, Z),
   course_path(Z, Y).
```

Query:

?- course_path(cs101, ai301).

Output:

true.

Facts

```
prerequisite(cs101, cs102).
prerequisite(cs102, cs201).
prerequisite(cs201, ai301).
```

What this mean:

These are facts. Each one states that:

- cs101 must be completed before taking cs102
- cs102 is a prerequisite for cs201
- cs201 is a prerequisite for ai301

So, the full chain is:

cs101 → cs102 → cs201 → ai301

Rules

Base Case (direct prerequisite):

course_path(X, Y) :- prerequisite(X, Y).

Explanation:

This defines a direct connection between two courses.
If X is a prerequisite for Y, then there's a course path from X to Y.

Query:

?- course_path(cs101, cs102).

true.

Recursive Case (indirect path):

```
course_path(X, Y) :-
  prerequisite(X, Z),
  course_path(Z, Y).
```

Explanation:

This says:

"There is a path from course X to course Y if:
X is a prerequisite for some Z, and there is a path from Z to Y (either direct or indirect)"

This allows Prolog to chain together multiple prerequisites — **that's what recursion does here.**

Query:

?- course_path(cs101, ai301).

Step-by-step:

Is cs101 a prerequisite for ai301 directly?

No.

Is cs101 a prerequisite for something (cs102)?

Yes → now check: course_path(cs102, ai301)

Is cs102 a prerequisite for something (cs201)?

Yes → now check: course_path(cs201, ai301)

Is cs201 a direct prerequisite for ai301?

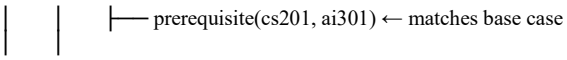
Yes → path exists!

Therefore:

course_path(cs101, ai301). → true

Recursive Call Tree

```
course_path(cs101, ai301)
│
├── prerequisite(cs101, cs102)
│   └── course_path(cs102, ai301)
│       ├── prerequisite(cs102, cs201)
│       │   └── course_path(cs201, ai301)
```



Explanation of Tree

Start at cs101 → check if ai301 is reachable
cs101 → cs102 → check if ai301 is reachable from cs102
cs102 → cs201 → check if ai301 is reachable from cs201
cs201 → ai301 → base case fires

So, the chain exists:

cs101 → cs102 → cs201 → ai301

Assignment 2:

Title: University Course and Student Management

Objective

To model a university system using Prolog that supports:
- Course enrollments
- Instructor assignments
- Prerequisite chains
- Eligibility checks
- Classmate and course path logic

Scenario

You are tasked with developing a Prolog-based knowledge system to track courses, students, prerequisites, and instructor assignments at a university. You must define a set of facts and rules that allow querying and inferring information about the academic structure.

Assignment Requirements

Part 1: Define Facts

You must include at least:
- 10 Students
- 6 Courses
- 3 Instructors
- 10 Enrollment records
- Prerequisite chain (for at least 4 courses)

Part 2: Write Rules

You must define rules for:

classmates(X, Y) — X and Y are enrolled in the same course.
teaches_course(Instructor, Course) — direct match (already covered).
is_eligible(Student, Course) — student is eligible if they've completed all prerequisites.
course_path(Start, End) — there is a path of prerequisites from Start to End.
students_of_instructor(Instructor, Student) — students enrolled in courses the instructor teaches.

Part 3: Sample Queries

Students must demonstrate these queries with correct results:

```
?- classmates(alice, bob).
?- is_eligible(alice, cs201).
?- course_path(cs101, ai301).
?- students_of_instructor(dr_kumar, X).
?- teaches(dr_smith, C), enrolled(bob, C).
```

Bonus Challenges (Optional)

Add support for grades and compute GPA
Add a drop course feature (via dynamic predicates)
Add major departments for students and filter eligible courses by major